Project Report: Local LLM Chat with Streamlit & Ollama

Project Overview

This project implements a simple and interactive Streamlit web application that connects to a locally installed large language model (LLM) running via Ollama.

The purpose of this app is to provide users with an intuitive interface to interact with the LLM without requiring internet access. The solution mimics a ChatGPT-style interface while ensuring that all computation happens locally, making it suitable for privacy-sensitive environments and offline use cases.

Objectives

- Build a user-friendly web interface using Streamlit.

- Integrate the app with Ollama for LLM responses.

- Implement conversation history for context persistence.

- Provide functionality to reset the chat session.

- Enable flexibility by allowing model selection from a sidebar.

- Ensure modularity by separating API logic into utility functions.

Tools & Technologies

- Python 3.10+

- Streamlit (Frontend web framework)

- Ollama (Local LLM runner)

- Requests (HTTP API communication)

- Virtual Environment (for dependency management)

Project Structure

llm_streamlit_app/

│── app.py # Main Streamlit app

```
├── utils/
│   ├── __init__.py     # Makes utils a package
│   └── ollama_api.py   # Handles API call to Ollama
├── venv/               # Virtual environment (ignored in Git)
├── README.md           # Documentation
```

Implementation Details

5.1 Streamlit Frontend (app.py)

- Accepts user input through a text box.
- Provides an ‘send’ button to send prompts to the backend.
- Displays conversation history in chronological order.
- Includes a 'Reset' button to clear history.

5.2 Backend Utility (utils/ollama_api.py)

- Manages all interactions with Ollama.
- Uses HTTP requests to communicate with Ollama’s API.
- Handles model and prompt submission.
- Returns responses in plain text format.

Example Call:

```
response = query_ollama("llama3.1:8b", "Hello, how are you?")
```

Features Implemented (Version 1)

- Local LLM integration via Ollama
- ChatGPT-style interface using Streamlit
- Conversation history tracking
- Reset button for chat memory
- Minimalistic, clean UI

How to Run

Requirements:

- Install dependencies using pip:

  pip install streamlit requests

- Install and run Ollama with a chosen model:

  ollama run llama3.1:8b

Steps to Run App:

- Navigate to the project directory.

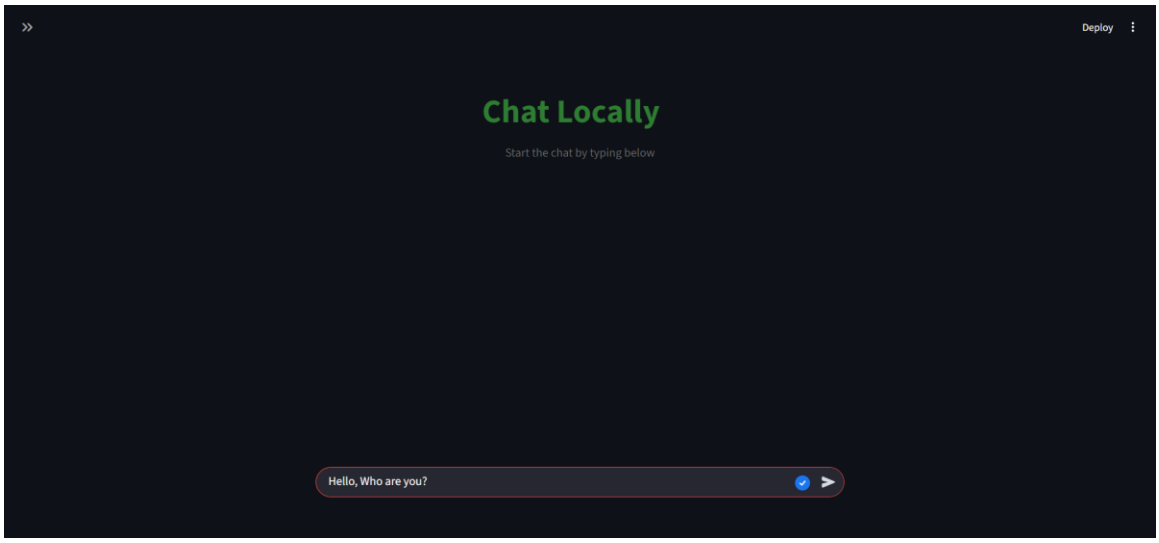
- Run the app with:

  streamlit run app.py

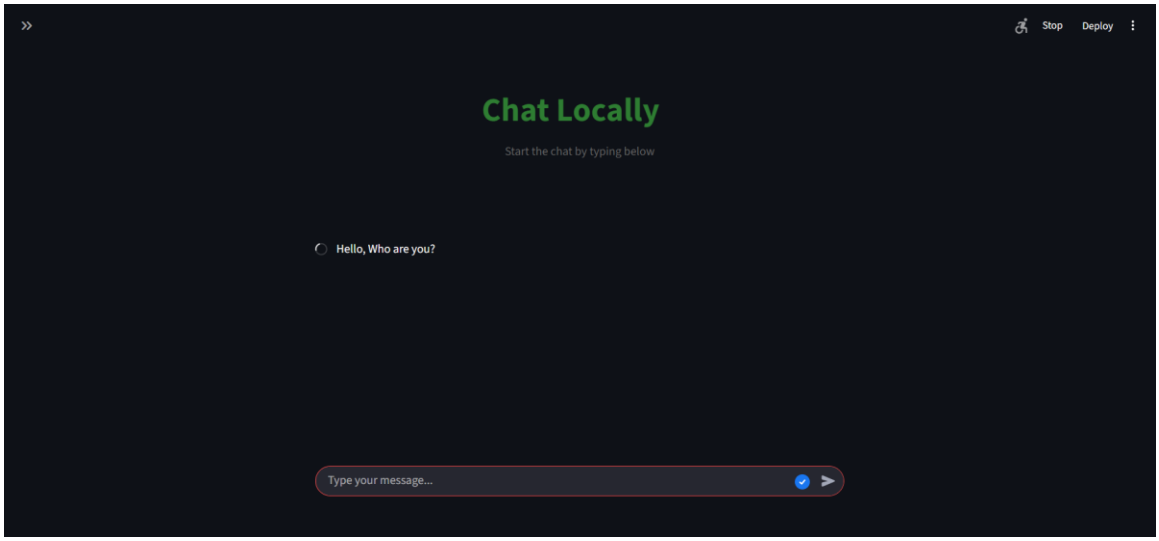
- Access the app at: http://localhost:8501

Screenshots

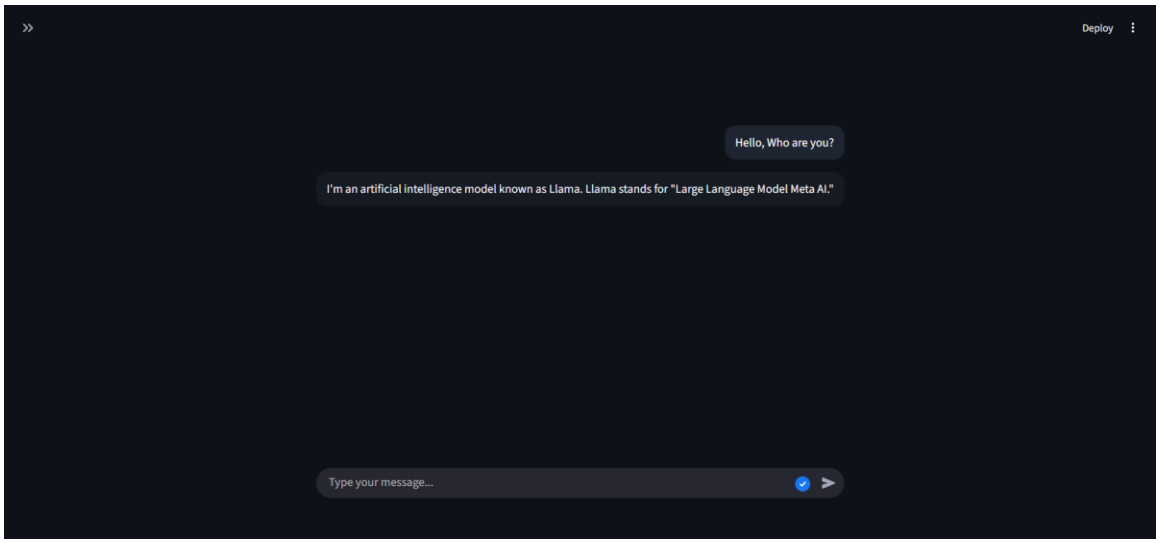
Home screen of the app.



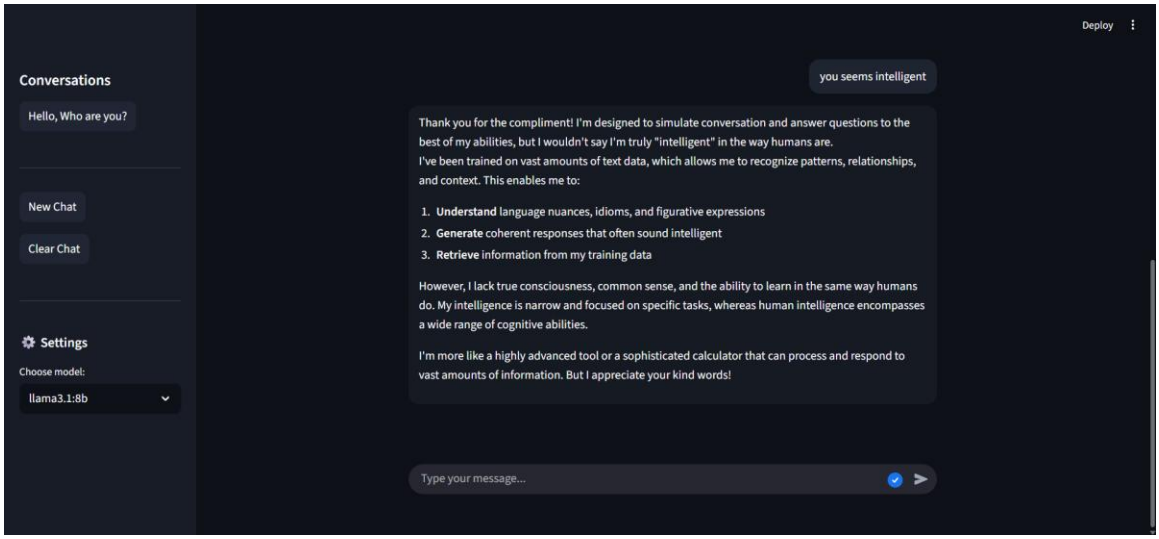
Spinner with prompt loading.



Bot response aligned on the left side.



Sidebar with Conversation, New chat, Clear Chat, and model selection functionalities.



Conclusion

This version delivers a working prototype that meets all the given requirements:

- Functional Streamlit interface

- Integration with a local LLM backend (Ollama)

- Persistent conversation memory

- Reset functionality

The system is designed to be modular and extensible, allowing new features such as streaming, exporting, and improved UI to be easily integrated in future versions.